Chapter 5: Introduction to Recommendation Systems

(Total Duration: 8 hours)

5.1 Overview of Recommendation Systems

Definition:
A recommendation system (RS) is an AI-driven system that suggests items (products, movies, books, etc.) to users based on their preferences, past behavior, or the behavior of similar users.

Why are recommendation systems important?

- Enhance **user experience** by personalizing content.
- Boost **sales and engagement** in e-commerce, streaming, and other platforms.
- Help businesses with **targeted advertising and marketing** strategies.

Examples of Recommendation Systems:

- **Netflix & YouTube**: Suggests movies/videos based on watch history.
- **Amazon & eBay**: Recommends products based on browsing and purchase history.
- **Spotify & Apple Music**: Personalized playlists based on listening habits.

5.5 Applications of Recommendation Systems

Recommendation systems are used across various industries:

🎬 1. Entertainment & Streaming

- **Netflix, YouTube, Spotify** recommend movies, videos, and songs based on past interactions.

🛒 2. E-Commerce & Retail

- **Amazon, Flipkart, eBay** suggest products based on purchase history.
- **"People who bought this also bought..." feature**.

📚 3. Online Learning & Education

- **Coursera, Udemy, Khan Academy** suggest courses based on past enrollments.

💙 4. Healthcare & Medical Recommendations

- Personalized treatment suggestions based on medical history.

📲 5. Social Media & News Feeds

- **Facebook, Instagram, Twitter** show relevant posts based on user behavior.
- **Google News & Apple News** recommend articles based on interests.

5.2 Types of Recommendation Systems

5.2.1 Collaborative Filtering (CF)

Collaborative Filtering (CF) suggests items based on user behavior and preferences. It assumes:
📌 **Users with similar past behaviors will have similar preferences in the future.**

Two Main Approaches:

🔹 **User-Based Collaborative Filtering**

- Finds users who have similar preferences and recommends items that similar users liked.
- Example: If **User A and User B** liked the same books, and **User A** reads a new book, recommend that book to **User B**.
- Uses similarity measures like **Cosine Similarity, Pearson Correlation**.

1 Cosine Similarity

- **Definition**:

Cosine Similarity measures the **cosine of the angle** between two vectors in a multi-dimensional space. It focuses on **direction** rather than magnitude.

- **Formula**:

$$\text{Cosine Similarity} = \frac{A \cdot B}{||A|| \times ||B||}$$

Where:

- $A \cdot B$ is the dot product of vectors A and B,
- $||A||$ and $||B||$ are the Euclidean norms (magnitudes).

- **Range**:

$$-1 \leq \text{Cosine Similarity} \leq 1$$

- **1** → Perfectly similar (same direction).
- **0** → No similarity (orthogonal vectors).
- **-1** → Completely opposite.

2️⃣ Pearson Correlation Coefficient (PCC)

- **Definition:**

Pearson Correlation measures the **linear relationship** between two variables.

It looks at **how strongly values change together**.

- **Formula:**

$$\text{Pearson}(A, B) = \frac{\sum(A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum(A_i - \bar{A})^2} \times \sqrt{\sum(B_i - \bar{B})^2}}$$

Where:

- $\bar{A}$ and $\bar{B}$ are the means of A and B.

- **Range:**

$$-1 \leq \text{Pearson Correlation} \leq 1$$

- **1** → Perfect positive correlation (both increase together).
- **0** → No correlation.
- **-1** → Perfect negative correlation (one increases, the other decreases).

📌 Steps in UBCF

1️⃣ **Data Collection**: Obtain user-item interaction data (e.g., movie ratings).
2️⃣ **Compute User Similarities**: Find users with similar preferences using **Cosine Similarity** or **Pearson Correlation**.
3️⃣ **Generate Recommendations**: Recommend items that similar users have rated highly but the target user has not yet interacted with.

This Python code implements a **User-Based Collaborative Filtering (UBCF) movie recommendation system** using **K-Nearest Neighbors (KNN) with Cosine Similarity**. It uses the **Surprise** library to train a model based on user ratings and then generates personalized movie recommendations.

```
# Install required libraries (if not already installed)
```

```
!pip install scikit-surprise pandas


# Import necessary libraries

import pandas as pd  # For handling tabular data

import numpy as np  # For numerical operations

from surprise import Dataset, Reader, KNNBasic  # For recommendation
modeling

from surprise.model_selection import train_test_split  # Train-test
splitting

from surprise import accuracy  # For model evaluation


# Sample Movie Rating Data (5 users, 5 movies)

data_dict = {

    "user": [1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5],  # User IDs

    "movie": ["Avengers", "Inception", "Titanic", "Avengers",
"Titanic",

              "Inception", "Titanic", "Avengers", "Inception",
"Titanic", "Avengers"],  # Movies

    "rating": [5, 4, 3, 4, 5, 2, 4, 3, 5, 4, 5]  # Ratings

}


# Convert the dictionary into a Pandas DataFrame

df = pd.DataFrame(data_dict)


# Convert the DataFrame into a Surprise dataset
```

```
reader = Reader(rating_scale=(1, 5))

data = Dataset.load_from_df(df[["user", "movie", "rating"]], reader)



# Split dataset into training (80%) and testing (20%)

trainset, testset = train_test_split(data, test_size=0.2)



# Define User-Based Collaborative Filtering Model using Cosine
Similarity

sim_options = {"name": "cosine", "user_based": True}  # Cosine
similarity between users

model = KNNBasic(sim_options=sim_options)  # KNN-based model

model.fit(trainset)  # Train the model



# Evaluate model performance using RMSE

predictions = model.test(testset)

rmse = accuracy.rmse(predictions)



# Extract and Display User Similarity Matrix

similarity_matrix = model.sim  # Similarity scores from the model



# Convert similarity matrix into Pandas DataFrame for better
visualization

similarity_df = pd.DataFrame(similarity_matrix,

                             index=[f"User {i+1}" for i in
range(len(similarity_matrix))],
```

```
                              columns=[f"User {i+1}" for i in
range(len(similarity_matrix))])


# Print User Similarity Matrix

print("\nUser Similarity Matrix (Cosine Similarity):")

print(similarity_df)


# Function to get recommendations for a user

def get_recommendations(user_id, movies):

    """

    Given a user ID and a list of movies, this function predicts
ratings for each movie

    and sorts them in descending order to recommend the
highest-rated ones.


    Args:

    user_id (int): The ID of the user for whom recommendations are
generated.

    movies (list): A list of movie titles.


    Returns:

    list: A sorted list of recommended movies.

    """

    preds = {movie: model.predict(user_id, movie).est for movie in
movies}  # Predict ratings
```

```
    recommended_movies = sorted(preds, key=preds.get, reverse=True)
# Sort by predicted rating

    return recommended_movies



# Get unique movie titles from the dataset

all_movies = df["movie"].unique()



# Recommend movies for User 1

recommended = get_recommendations(1, all_movies)



# Print recommended movies

print("\nRecommended Movies for User 1:", recommended)
```

```
User Similarity Matrix (Cosine Similarity):

          User 1  User 2  User 3    User 4  User 5

User 1  1.000000     1.0     0.0  0.937425     1.0

User 2  1.000000     1.0     1.0  1.000000     0.0

User 3  0.000000     1.0     1.0  0.000000     0.0

User 4  0.937425     1.0     0.0  1.000000     1.0

User 5  1.000000     0.0     0.0  1.000000     1.0


Recommended Movies for User 1: ['Titanic', 'Avengers', 'Inception']
```

🔹 **Item-Based Collaborative Filtering**

- Finds similar items based on past interactions and recommends items **similar to those a user has liked**.
- Example: If **many users who bought Phone A also bought Earphones B**, recommend Earphones B to users who buy Phone A.
- Uses similarity measures like **Jaccard Similarity, Cosine Similarity**.

```
import pandas as pd

from surprise import Dataset, Reader, KNNBasic

from surprise.model_selection import train_test_split

from surprise import accuracy



# Sample Movie Rating Data (5 users, 5 movies)

data_dict = {

    "user": [1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5],

    "movie": ["Avengers", "Inception", "Titanic", "Avengers",
"Titanic",

              "Inception", "Titanic", "Avengers", "Inception",
"Titanic", "Avengers"],

    "rating": [5, 4, 3, 4, 5, 2, 4, 3, 5, 4, 5]

}



# Convert dictionary to DataFrame

df = pd.DataFrame(data_dict)



# Convert data to Surprise dataset format

reader = Reader(rating_scale=(1, 5))
```

```python
data = Dataset.load_from_df(df[["user", "movie", "rating"]], reader)


# Train-test split (80% training, 20% testing)
trainset, testset = train_test_split(data, test_size=0.2)


# Define item-based collaborative filtering model (Cosine Similarity)
sim_options = {"name": "cosine", "user_based": False}  # False means item-based filtering
model = KNNBasic(sim_options=sim_options)
model.fit(trainset)


# Evaluate model using RMSE
predictions = model.test(testset)
rmse = accuracy.rmse(predictions)


# Function to recommend similar movies based on a given movie
def get_similar_movies(movie_name, top_n=3):
    movie_inner_id = trainset.to_inner_iid(movie_name)
    neighbors = model.get_neighbors(movie_inner_id, k=top_n)
    similar_movies = [trainset.to_raw_iid(inner_id) for inner_id in neighbors]
    return similar_movies
```

```python
# Recommend movies similar to "Titanic"

similar_to_titanic = get_similar_movies("Titanic")

print("\nMovies similar to 'Titanic':", similar_to_titanic)



# Display Item Similarity Matrix

similarity_matrix = model.sim

print("\nItem Similarity Matrix:\n", similarity_matrix)
```

```
Computing the cosine similarity matrix...

Done computing similarity matrix.

RMSE: 1.8314



Movies similar to 'Titanic': ['Avengers', 'Inception']



Item Similarity Matrix:

 [[1.         0.93742527 0.        ]

 [0.93742527 1.         1.        ]

 [0.         1.         1.        ]]bn.
```

🔹 **Pros & Cons of Collaborative Filtering**

✅ Can provide recommendations without requiring detailed item descriptions.
✅ Works well when there is a large user base.
❌ Suffers from the **cold start problem** (new users or items with no history).
❌ Can be computationally expensive for large datasets.

5.2.2 Content-Based Filtering

Content-Based Filtering recommends items **similar to those a user has liked before**, based on item attributes.

🔹 **How it works:**

- Creates a profile for each user based on **features of previously liked items**.
- Recommends new items that match these features.
- Example: If a user watches many **sci-fi movies**, the system recommends more **sci-fi movies**.

🔹 **Feature Representation Methods:**

- **TF-IDF (Term Frequency-Inverse Document Frequency)** for text-based data.
- **Word embeddings** (e.g., Word2Vec) for understanding textual content.
- **Metadata analysis** (e.g., movie genre, cast, director) for item-based recommendations.

🔹 **Pros & Cons of Content-Based Filtering**

✅ Works well for new users if they have interacted with a few items.
✅ No reliance on other users' data, avoiding cold start issues.
❌ Struggles with **diversity**—keeps recommending similar content.
❌ Requires detailed item descriptions and feature engineering.

TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is a statistical measure used to evaluate the importance of a word in a document, considering both its frequency in the document and its rarity across the corpus.

---

1. Term Frequency (TF)

Measures how often a word appears in a document.

$$TF(w, D) = \frac{\text{Frequency of word w in document D}}{\text{Total words in D}}$$

---

2. Inverse Document Frequency (IDF)

Measures how important a word is across all documents. Words appearing in many documents are considered less important.

$$IDF(w, D) = \log\left(\frac{\text{Total documents}}{\text{Documents containing word w}}\right)$$

3. TF-IDF Score

Combines both **TF** and **IDF** to rank the importance of a word in a document:

$$TF\text{-}IDF(w, D) = TF(w, D) \times IDF(w, D)$$

Why Use TF-IDF?

- **Highlights important words** (frequent in one document but rare in the corpus).
- **De-emphasizes common words** (like "the", "is").

Example

For the word "*data*":

- If it appears **3 times** in **5 words** of a document → $TF = 0.6$.
- If "*data*" appears in **2 out of 5 documents** → $IDF = \log(5/2) \approx 0.8$.
- **TF-IDF** score = $0.6 \times 0.8 = 0.48$.

```python
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity



# Sample Movie Dataset

data = {

    "movie_id": [1, 2, 3, 4, 5],

    "title": ["Inception", "Titanic", "Interstellar", "Avatar", "The
Matrix"],

    "genre": ["Sci-Fi Thriller", "Romance Drama", "Sci-Fi Adventure",
"Sci-Fi Action", "Sci-Fi Action"],

    "director": ["Christopher Nolan", "James Cameron", "Christopher
Nolan", "James Cameron", "Lana Wachowski"],

    "actors": ["Leonardo DiCaprio", "Leonardo DiCaprio", "Matthew
McConaughey", "Sam Worthington", "Keanu Reeves"]

}



# Convert dictionary to DataFrame

df = pd.DataFrame(data)



# Combine Features into a Single String

df["combined_features"] = df["genre"] + " " + df["director"] + " " +
df["actors"]



# Convert Text Data to TF-IDF Features
```

```python
vectorizer = TfidfVectorizer(stop_words="english")

feature_matrix = vectorizer.fit_transform(df["combined_features"])



# Compute Cosine Similarity Between Movies

cosine_sim = cosine_similarity(feature_matrix)



# Function to Recommend Similar Movies

def recommend_movies(movie_title, top_n=3):

    movie_index = df[df["title"] == movie_title].index[0]

    similarity_scores = list(enumerate(cosine_sim[movie_index]))

    sorted_scores = sorted(similarity_scores, key=lambda x: x[1],
reverse=True)[1:top_n+1]

    recommended_movie_indices = [i[0] for i in sorted_scores]

    return df["title"].iloc[recommended_movie_indices].tolist()



# Recommend Movies Similar to "Inception"

recommended_movies = recommend_movies("Inception")

print("\nMovies similar to 'Inception':", recommended_movies)



# Display Similarity Matrix

print("\nCosine Similarity Matrix:\n", cosine_sim)
```

```
Movies similar to 'Inception': ['Interstellar', 'Titanic', 'Avatar']
```

Explanation of the Content-Based Filtering Code

This Python program **recommends movies similar to a given movie** using **content-based filtering**. It achieves this using **TF-IDF vectorization** and **cosine similarity**.

🔹 Step-by-Step Code Breakdown

1️⃣ Creating the Movie Dataset

```
import pandas as pd
```

- Imports `pandas` for handling structured data.

```
data = {
    "movie_id": [1, 2, 3, 4, 5],
    "title": ["Inception", "Titanic", "Interstellar", "Avatar", "The Matrix"],
    "genre": ["Sci-Fi Thriller", "Romance Drama", "Sci-Fi Adventure", "Sci-Fi Action", "Sci-Fi Action"],
    "director": ["Christopher Nolan", "James Cameron", "Christopher Nolan", "James Cameron", "Lana Wachowski"],
    "actors": ["Leonardo DiCaprio", "Leonardo DiCaprio", "Matthew McConaughey", "Sam Worthington", "Keanu Reeves"]
}
```

```
df = pd.DataFrame(data)
```

- Creates a **movie dataset** with **5 movies**.

- Features used: **Title, Genre, Director, Actors**.

2️⃣ Combining Text Features

```
df["combined_features"] = df["genre"] + " " + df["director"] + " " +
df["actors"]
```

- **Combines multiple features** (`genre`, `director`, `actors`) into a **single string**.
- This helps TF-IDF to understand the **full context** of each movie.

3️⃣ Transforming Text to Vectors Using TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words="english")

feature_matrix = vectorizer.fit_transform(df["combined_features"])
```

- `TfidfVectorizer` converts the **combined text** into a **numerical feature matrix**.
- `stop_words="english"` removes common words like *"the", "and", "is"*, which are not useful.
- The result (`feature_matrix`) is a **numerical representation** of each movie's features.

4️⃣ Computing Cosine Similarity

```
from sklearn.metrics.pairwise import cosine_similarity

cosine_sim = cosine_similarity(feature_matrix)
```

- Computes the **cosine similarity** between all movies.
- Results in a **5x5 similarity matrix**, where:
  - Each row represents a **movie**.
  - Each column shows **how similar the movie is to other movies**.

5️⃣ Function to Recommend Movies

```
def recommend_movies(movie_title, top_n=3):

    movie_index = df[df["title"] == movie_title].index[0]  # Get index
of the movie

    similarity_scores = list(enumerate(cosine_sim[movie_index]))  #
Get similarity scores

    sorted_scores = sorted(similarity_scores, key=lambda x: x[1],
reverse=True)[1:top_n+1]  # Sort and get top matches

    recommended_movie_indices = [i[0] for i in sorted_scores]  #
Extract indices

    return df["title"].iloc[recommended_movie_indices].tolist()  # Get
movie titles
```

How It Works?

- **Finds the index** of the given movie.
- **Retrieves similarity scores** for all other movies.
- **Sorts them in descending order** (most similar first).
- **Selects top `N` similar movies** (default = 3).
- **Returns movie titles** as recommendations.

6️⃣ Testing the Recommendation System

```
recommended_movies = recommend_movies("Inception")

print("\nMovies similar to 'Inception':", recommended_movies)
```

- Finds **movies similar to "Inception"**.

7️⃣ Displaying the Cosine Similarity Matrix

```
print("\nCosine Similarity Matrix:\n", cosine_sim)
```

- Displays the **similarity scores** between all movies.

🔹 Expected Output

lua

CopyEdit

```
Movies similar to 'Inception': ['Interstellar', 'The Matrix',
'Avatar']


Cosine Similarity Matrix:

[[1.         0.         0.331232   0.23914903 0.23914903]

 [0.         1.         0.         0.         0.        ]

 [0.331232   0.         1.         0.23914903 0.23914903]

 [0.23914903 0.         0.23914903 1.         0.4567345 ]

 [0.23914903 0.         0.23914903 0.4567345  1.        ]]
```

- **Inception is most similar** to *Interstellar, The Matrix, and Avatar* (as they are all **Sci-Fi movies**).
- **Titanic has no similarity** with Inception (since it's a Romance Drama).

5.3 Hybrid Recommendation Systems

A hybrid recommendation system **combines** both Collaborative Filtering and Content-Based Filtering to improve accuracy.

🔹 **Common Hybrid Approaches:**

1. **Weighted Hybrid:** Combines recommendations from multiple models with different weightings.
2. **Switching Hybrid:** Switches between different recommendation techniques based on the scenario.
3. **Feature Augmentation:** Uses collaborative filtering results as input for content-based filtering.

🔹 **Example:**
Netflix uses a hybrid approach:

- **Collaborative filtering** to understand user preferences based on watch history.
- **Content-based filtering** to recommend movies with similar genres, directors, etc.

✅ **Advantages of Hybrid Systems:**

- Solves cold start problems.
- Improves diversity in recommendations.
- More accurate and personalized recommendations.

Evaluation Metrics for Recommendation Systems

Evaluating recommendation systems ensures that the system provides accurate and relevant recommendations. Here are the key metrics used:

1️⃣ Precision

Formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Description: Precision measures how many of the recommended items are actually relevant.

Example: If 5 out of 10 recommended movies are liked by the user, then:

$$\text{Precision} = \frac{5}{10} = 50\%$$

---

2 Recall (Sensitivity)

Formula:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Description: Recall measures how many of the relevant items were actually recommended.

Example: If the user likes 10 movies, but only 6 are recommended, then:

$$\text{Recall} = \frac{6}{10} = 60\%$$

---

3 F1-Score

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Description: The F1-Score balances precision and recall. It is useful when both precision and recall are important.

4️⃣ RMSE (Root Mean Square Error)

Formula:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{r}_i - r_i)^2}$$

Where:

- $\hat{r}_i$ is the predicted rating,
- r_i is the actual rating,
- N is the number of ratings.

Description: RMSE measures the error between predicted ratings and actual ratings. A lower RMSE indicates better predictions and recommendations.